

CI/CD Pipeline Standards

wiki/engineering/cicd-pipeline · Updated 2026-08-30

Owner: Platform Engineering · POC: Marcus Lee, Platform Lead

Purpose

This document defines the standard continuous integration and continuous delivery (CI/CD) process for all Northwind services. It is the single source of truth for how code moves from a pull request to production. Every engineering team is expected to follow it.

The goals of the pipeline are safety, repeatability, and auditability. No change should reach customers without passing automated checks, and every production change must be traceable to a merged pull request and a green pipeline run.

Principles

- Every change is reviewed and tested before it ships.
- The pipeline, not a person, performs deploys.
- What runs in production always matches a reviewed commit.
- Rollbacks are fast, boring, and always available.
- Secrets never live in the repository or in build logs.

Pipeline stages

All deployments go through GitHub Actions. On merge to main, the workflow builds a container and deploys it to Cloud Run automatically. The pipeline runs the following stages in order, and a failure in any stage stops the release:

- Lint and static analysis on every pull request.
- Unit and integration tests, with coverage reported back to the pull request.
- Container build, tagged with the commit SHA and pushed to Artifact Registry.
- Automated deploy to the staging Cloud Run service.
- Smoke tests against staging.
- Promotion to the production Cloud Run service on merge to main.

Pull request checks

Every pull request triggers the lint and test stages. The results are reported directly on the pull request so reviewers can see them. A pull request cannot merge until all required checks pass.

Build and artifact storage

On merge, the pipeline builds a container image tagged with the commit SHA and pushes it to Artifact Registry. Images are immutable and retained, which is what makes fast rollbacks possible.

Branch protection

The main branch is protected. Pull requests require at least one approval and a fully green pipeline before they can merge. Required status checks cannot be bypassed, and force-pushes to main are disabled.

No manual deploys

Manual deployment is forbidden: the legacy `deploy.sh` script was removed in 2025 and must never be used. Any service still referencing it must be migrated to the pipeline before its next release. If you find documentation or runbooks that tell you to deploy by hand, treat them as out of date and raise a correction.

The only supported way to ship a change is to merge an approved pull request to main and let the pipeline promote the build. This guarantees that what runs in production exactly matches a reviewed, tested commit.

Why manual deploys were removed

Manual deploys bypassed review and testing, produced releases that did not match any reviewed commit, and left no reliable audit trail. Removing them eliminated an entire category of production incidents caused by someone shipping an unreviewed or stale build.

Environments

There are two deploy environments: staging and production, each a separate Cloud Run service. Changes are always promoted to staging first and smoke-tested there before production. Never test an unverified change directly in production.

Rollbacks

Rollbacks are handled by redeploying a previous Cloud Run revision from the console. Because every revision is immutable and retained, rolling back is a single action and does not require a rebuild. Pin traffic to the last known-good revision, confirm the service recovers, then open an incident to track the root cause.

Gradual rollout

For higher-risk changes, use Cloud Run traffic splitting to send a small percentage of requests to the new revision first. Watch the metrics, then shift traffic to 100 percent once the new revision proves healthy. If the new revision misbehaves, shift traffic back to the previous revision instantly.

Secrets and configuration

Pipeline secrets live in Secret Manager and are injected at deploy time.

Never commit secrets to the repository or print them in build logs.

Configuration that differs between staging and production is managed through environment variables set on each Cloud Run service, not baked into the container image.

Managing configuration changes

Treat configuration changes with the same care as code changes. A bad environment variable can be just as damaging as a bad commit. Review configuration changes and roll them out through the same promotion path where possible.

Worked example

An engineer opens a pull request. The pipeline lints and tests it and reports green on the pull request. A reviewer approves. On merge to main, the pipeline builds a container, pushes it to Artifact Registry, deploys to staging, runs smoke tests, and promotes the build to production on Cloud Run. If the production revision shows errors, the engineer pins traffic back to the previous revision from the console and opens an incident.

Frequently asked questions

Can I bypass a failing check in an emergency? No. Required checks exist precisely for emergencies. If a check is wrong, fix the check.

How do I roll back? Redeploy the previous Cloud Run revision from the console and pin traffic to it. No rebuild is needed.

What happened to `deploy.sh`? It was removed in 2025. Manual deploys are forbidden; use the pipeline.

Common mistakes

- Trying to deploy by hand with an old script instead of using the pipeline.
- Committing a secret and then rewriting history to hide it instead of rotating it.
- Baking environment-specific config into the container image.
- Merging with a red pipeline by disabling a required check.

Ownership

The Platform team owns the pipeline definition and the shared workflow templates. Service teams own their own tests and deploy configuration. Requests to change the shared workflow go through the Platform team's review.

Related documents

See the Deployment Guide for the service-level view, the On-Call Runbook for incident response, and the API Authentication Guide for how services handle secrets.